Conda + uv Workflow Guide (Full Monty) Miniconda (conda commands) macOS & WSL2 (CUDA)

This guide documents a **stable, reproducible Python workflow** using: **Miniconda** (via conda) for Python 3.13, JupyterLab, CUDA, PyTorch, and core tooling **uv** for domain-specific Python dependency management and lockfiles Recommended version-controlled artifacts: environment.yaml — conda-managed runtime requirements pyproject.toml and uv.lock — uv-managed domain requirements

Optional: Enable the libmamba Solver (Recommended)

```
conda config --set solver libmamba
```

Part 1 — macOS Environment (CPU / Apple Silicon MPS)

Create Environment

```
conda create -n myenv-mac python=3.13 jupyterlab ipykernel wheel setuptools
```

```
conda activate myenv-mac
```

Install PyTorch (CPU / MPS)

```
conda install pytorch -c pytorch
```

Verify PyTorch

```
python - <<'EOF' import torch print(torch.__version__) print('MPS available:', torch.backends.mps.is_available()) EOF
```

Export Conda Environment (Recommended)

Export a clean environment.yaml capturing only explicitly-installed conda packages (**excluding** uv/pip domain dependencies):

```
conda env export --from-history > environment.yaml
```

Initialize Project with uv

```
cd your_project_directory
```

```
uv init
```

```
uv python pin "$(which python)"
```

```
uv add numpy pandas scipy # add your domain packages here
```

```
uv lock
```

```
uv sync
```

Register Kernel & Launch JupyterLab

```
python -m ipykernel install --user --name myenv-mac --display-name "Python (myenv-mac)"
```

```
jupyter lab
```

Part 2 — Windows WSL2 Environment (CUDA GPU)

Verify GPU Access (inside WSL2)

```
nvidia-smi
```

Create Environment

```
conda create -n myenv-wsl python=3.13 jupyterlab ipykernel wheel setuptools
```

```
conda activate myenv-wsl
```

Install CUDA-enabled PyTorch (Recommended via conda)

```
conda install pytorch pytorch-cuda=12.1 -c pytorch -c nvidia
```

Verify CUDA

```
python - <<'EOF' import torch print(torch.__version__) print('CUDA available:',
torch.cuda.is_available()) print('CUDA:', torch.version.cuda) EOF
```

Export Conda Environment (Recommended)

```
conda env export --from-history > environment.yaml
```

Initialize Project with uv (Domain packages only)

In CUDA environments, do not install torch via uv/pip—let conda own torch + CUDA.

```
cd your_project_directory
```

```
uv init
```

```
uv python pin "$(which python)"
```

```
uv add numpy pandas scipy # add your domain packages here
```

```
uv lock
```

```
uv sync
```

Register Kernel & Launch JupyterLab

```
python -m ipykernel install --user --name myenv-wsl --display-name "Python
(myenv-wsl)"
```

```
jupyter lab
```

Appendix A — One-Page Quick Start

macOS: conda create -n myenv-mac python=3.13 jupyterlab ipykernel conda activate myenv-mac conda install pytorch -c pytorch Verify torch (MPS optional) conda env export --from-history > environment.yaml uv init && uv python pin $(which python) uv add <domain packages> && uv sync python -m ipykernel install --user --name myenv-mac jupyter lab

WSL2 CUDA: nvidia-smi conda create -n myenv-wsl python=3.13 jupyterlab ipykernel conda activate myenv-wsl conda install pytorch pytorch-cuda=12.1 -c pytorch -c nvidia Verify torch.cuda.is_available() conda env export --from-history > environment.yaml uv init && uv python pin $(which python) uv add <domain packages> && uv sync python -m ipykernel install --user --name myenv-wsl jupyter lab

Appendix B — Project Directory Template

```
project-root/ ■■ environment.yaml ■■ pyproject.toml ■■ uv.lock ■■ README.md ■■ notebooks/ ■ ■■ exploration.ipynb ■■ src/ ■ ■■ your_package/ ■ ■■ __init__.py ■ ■■ core.py ■■ scripts/ ■ ■■ export_env.sh ■ ■■ train.py ■■ data/ ■■ raw/ ■■ processed/
```

Notes: Commit environment.yaml, pyproject.toml, and uv.lock Do not commit conda environments Keep environment export scripts under scripts/

Appendix C — Safe environment.yaml Regeneration Script

Use the following script to (re)generate environment.yaml safely. It: Requires an active conda environment Exports using --from-history to avoid uv/pip pollution Backs up any existing environment.yaml with a timestamp Optionally records the active environment name Save as scripts/export_env.sh and run it from your project root.

```
#!/usr/bin/env bash set -euo pipefail # scripts/export_env.sh # Regenerate environment.yaml from the *currently active* conda env, # capturing only explicitly installed conda packages. OUT_FILE="${1:-environment.yaml}" if [[ -z "${CONDA_PREFIX:-}" ]]; then echo "ERROR: No active conda environment detected. Run: conda activate <env>" >&2 exit 1 fi ENV_NAME="${CONDA_DEFAULT_ENV:-unknown-env}" TS="$(date +%Y%m%d-%H%M%S)" if [[ -f "$OUT_FILE" ]]; then cp "$OUT_FILE" "${OUT_FILE}.${TS}.bak" echo "Backed up existing $OUT_FILE -> ${OUT_FILE}.${TS}.bak" fi # Export only explicitly installed packages (clean for conda+uv workflows) conda env export --from-history > "$OUT_FILE" # Optionally annotate the environment name at the top (YAML comment) # This does not affect conda env create, but helps humans. sed -i.bak "1s/^/# Exported from: ${ENV_NAME} at ${TS}\n/" "$OUT_FILE" 2>/dev/null || true rm -f "${OUT_FILE}.bak" 2>/dev/null || true echo "Wrote $OUT_FILE (from-history) from active env: $ENV_NAME"
```

Usage: conda activate myenv From project root: bash scripts/export_env.sh Optional output path: bash scripts/export_env.sh env/environment.yaml

Appendix D — Troubleshooting

Notebook can't import packages installed by uv

Ensure the notebook kernel matches the conda env (Kernel → Change Kernel). Re-pin uv: uv python pin $(which python)

uv installs to the wrong interpreter

Activate conda env first. Then run uv python pin $(which python) and re-run uv sync.

WSL2 CUDA not available

Confirm nvidia-smi works inside WSL2 and that you installed pytorch-cuda via conda. Avoid pip/uv torch installs in CUDA envs.

Recreate a working environment

conda env create -f environment.yaml conda activate <env> uv sync